

```
In [1]: import re
import os
import matplotlib.pyplot as plt
import numpy as np
import contextily as ctx
from shapely import wkt
from shapely.wkt import loads
from shapely.geometry import mapping
from shapely.ops import unary_union
import geopandas as gpd
import pandas as pd
import h5py
import s3fs
import asf_search as asf
import earthaccess
from rasterio.crs import CRS
from rasterio.transform import Affine
import rasterio
import json

# from openseppo.cli import nisar_search
```

```
In [2]: # parse the filename and the query, to get some information about the
def parse_nisar_filename(fileID, s3Urls):
    parts = fileID.split('_')
    date = next(p[:8] for p in parts if re.match(r'\d{8}T\d{6}', p))
    h5_url = next((u for u in s3Urls if re.search(r'\d+\.\h5$', u)), None)
    return {
        'product': parts[3],
        'track': parts[5],
        'orb': parts[6],
        'frame': parts[7],
        'date': date,
        'proc': parts[13],
        's3_url': h5_url,
    }
```

```
In [3]: # area of interest for querying data
west, east, south, north = -83.1, -83.0, 29.8, 29.9
aoi_search = f'POLYGON(({west} {south}, {east} {south}, {east} {north}

aois = {
    'Gainesville': 'POLYGON((-82.2 29.3, -81.7 29.3, -81.7 29.7, -82.2
}
aois = {
    'Gainesville': 'POLYGON((-82.4 29.4, -81.7 29.4, -81.7 29.8, -82.4
}

results = asf.geo_search(platform=[asf.PLATFORM.NISAR], intersectsWith
if False:
```

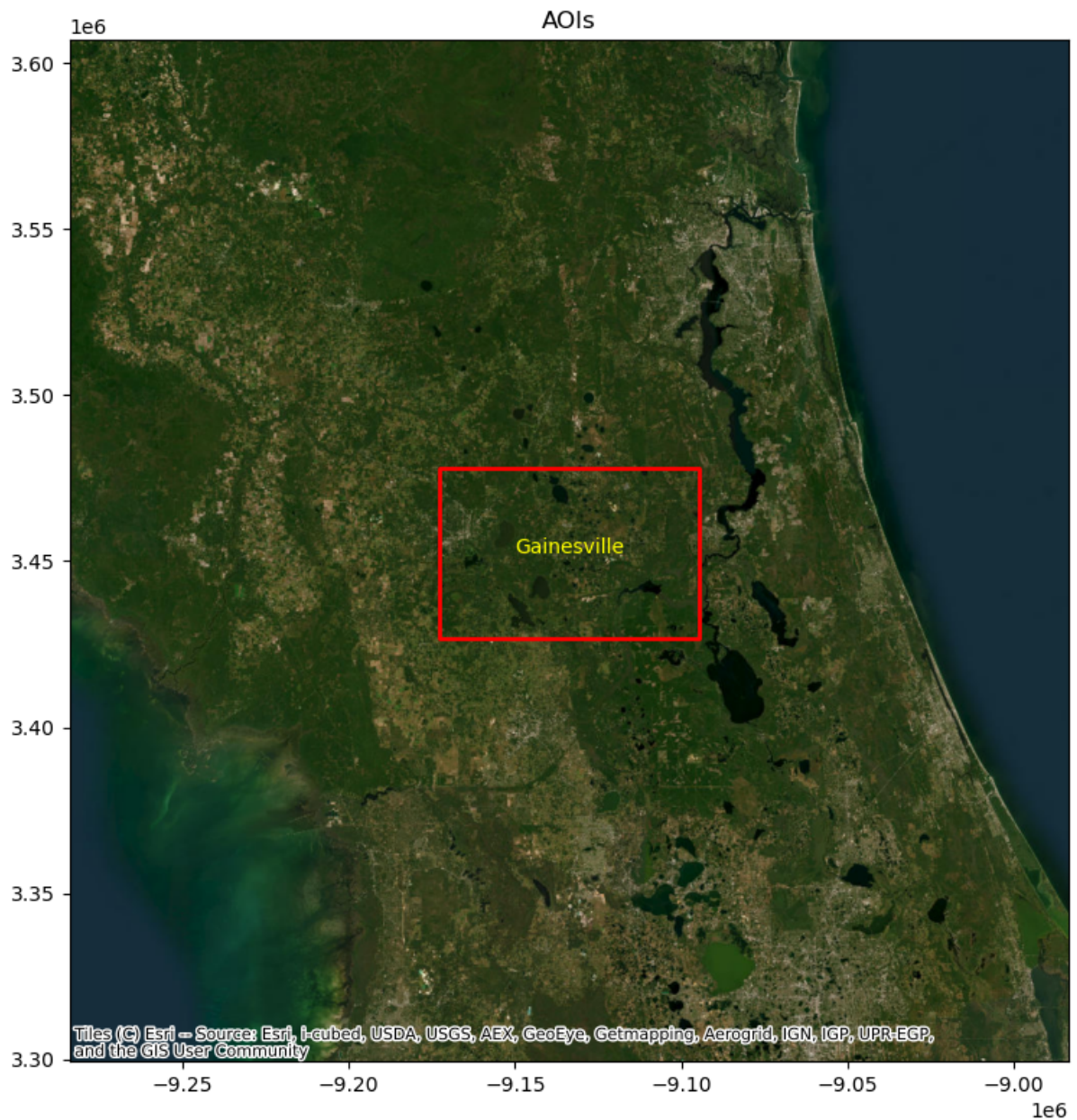
```
print(json.dumps(results[0].properties, indent=2, default=str))
```

```
In [4]: # make an optical image of the region along with the area of interest
#
# EPSG 4326 is lat/lon (deg)
# EPSG 3857 is Web Mercator (meters)
geoms = [loads(wkt_str) for wkt_str in aois.values()]
all_geom = unary_union(geoms)
gdf = gpd.GeoDataFrame(geometry=geoms, crs='EPSG:4326')
gdf['name'] = list(aois.keys())
gdf_web = gdf.to_crs(epsg=3857)

buf = 1.0 # buffer around the area of interest (units of degrees)
buffered = gpd.GeoDataFrame(geometry=[all_geom.buffer(buf)], crs='EPSG

fig, ax = plt.subplots(figsize=(8, 8))
gdf_web.plot(ax=ax, facecolor='none', edgecolor='red', linewidth=2)
for _, r in gdf_web.iterrows():
    ax.annotate(r['name'], xy=r.geometry.centroid.coords[0], ha='cente

minx, miny, maxx, maxy = buffered.total_bounds
ax.set_xlim(minx, maxx)
ax.set_ylim(miny, maxy)
ctx.add_basemap(ax, source=ctx.providers.Esri.WorldImagery)
plt.title('AOIs')
plt.tight_layout()
plt.show()
```



```
In [5]: # summary of the data found
# some filtering is used to make it so that there are only ascending o
df = pd.DataFrame([
    parse_nisar_filename(r.properties['fileID'], r.properties['s3Urls'
    for r in results
    if parse_nisar_filename(r.properties['fileID'], r.properties['s3U
])
df = df[df['orb'] == 'D']
df = df.sort_values('date')
print(df.drop(columns=['s3_url']).to_string())
```

	product	track	orb	frame	date	proc
15	GCOV	026	D	073	20251112	X05010
12	GCOV	098	D	074	20251129	X05009
11	GCOV	098	D	074	20251211	X05009
8	GCOV	026	D	073	20251218	X05009
6	GCOV	098	D	074	20251223	X05009
3	GCOV	026	D	073	20251230	X05009
2	GCOV	098	D	074	20260104	X05010
1	GCOV	026	D	073	20260111	X05010
0	GCOV	098	D	074	20260116	X05010

```
In [6]: # create credentials for accessing the NISAR data
auth = earthaccess.login()
endpoint = 'https://nisar.asf.earthdatacloud.nasa.gov/s3credentials'
s3_credentials = auth.get_s3_credentials(endpoint=endpoint)

fs = s3fs.S3FileSystem(
    anon=False,
    key=s3_credentials['accessKeyId'],
    secret=s3_credentials['secretAccessKey'],
    token=s3_credentials['sessionToken']
)
```

```
In [7]: # for each file, go to the s3 bucket and query the projection, bounding box
# keep only those track/frames that have the same projection.
# find the overall bounding box and store that in units of km

EPSG_keep = 32617
epsg_string = f"EPSG:{EPSG_keep}"

bb_list = [] # list of bounding boxes
proj_list = [] # list of projections for each track/frame
keep_mask = [] # list of which track/frames to keep

for idx, row in df.iterrows():
    h5file = row['s3_url']
    with fs.open(h5file, 'rb') as f:
        with h5py.File(f, 'r') as h5:
            proj = h5['science/LSAR/GCOV/grids/frequencyA/projection']
            passes = (proj == EPSG_keep)
            keep_mask.append(passes)
            if passes:
                bb = wkt.loads(h5['science/LSAR/GCOV/metadata/ceos'])
                xdelta = h5['science/LSAR/GCOV/grids/frequencyA/xCoord']
                ydelta = h5['science/LSAR/GCOV/grids/frequencyA/yCoord']
                bb_list.append(bb)
                proj_list.append(proj)
                print(f"{idx:02d} {bb}\tEPSG: {proj}\tdx={xdelta:.1f}")

df = df[keep_mask].reset_index(drop=True)
bb_total = unary_union(bb_list)
minx, miny, maxx, maxy = np.array(bb_total.bounds) / 1000.0
```

```

print(f"(minx, miny, maxx, maxy): ({minx:.2f}, {miny:.2f}, {maxx:.2f},
12 POLYGON Z ((203760 3348720 0, 203760 3006000 0, 552960 3006000 0, 5
52960 3348720 0, 203760 3348720 0))      EPSG: 32617      dx=10.0, dy=-1
0.0
11 POLYGON Z ((203760 3348720 0, 203760 3006000 0, 552960 3006000 0, 5
52960 3348720 0, 203760 3348720 0))      EPSG: 32617      dx=10.0, dy=-1
0.0
06 POLYGON Z ((203760 3348720 0, 203760 3006000 0, 552960 3006000 0, 5
52960 3348720 0, 203760 3348720 0))      EPSG: 32617      dx=10.0, dy=-1
0.0
02 POLYGON Z ((203760 3348720 0, 203760 3006000 0, 552960 3006000 0, 5
52960 3348720 0, 203760 3348720 0))      EPSG: 32617      dx=10.0, dy=-1
0.0
00 POLYGON Z ((203760 3348720 0, 203760 3006000 0, 552960 3006000 0, 5
52960 3348720 0, 203760 3348720 0))      EPSG: 32617      dx=10.0, dy=-1
0.0
(minx, miny, maxx, maxy): (203.76, 3006.00, 552.96, 3348.72)

```

```

In [8]: stride = 1
all_subsets = {}

for aoi_name, aoi_wkt in aois.items():
    aoi_gdf = gpd.GeoDataFrame(geometry=[loads(aoi_wkt)], crs='EPSG:43
    aoi_xmin, aoi_ymin, aoi_xmax, aoi_ymax = aoi_gdf.geometry[0].bound
    print(f"\nAOI '{aoi_name}' in {epsg_string}: x=[{aoi_xmin:.0f}, {a

subsets = {}
for idx, row in df.iterrows():
    h5file = row['s3_url']
    key = f"{row['date']}_t{row['track']}_f{row['frame']}"
    with fs.open(h5file, 'rb') as f:
        with h5py.File(f, 'r') as h5:
            xCoords = h5['/science/LSAR/GCOV/grids/frequencyA/xCoo
            yCoords = h5['/science/LSAR/GCOV/grids/frequencyA/yCoo

            xi0 = np.searchsorted(xCoords, aoi_xmin)
            xi1 = np.searchsorted(xCoords, aoi_xmax)
            yi0 = np.searchsorted(-yCoords, -aoi_ymax)
            yi1 = np.searchsorted(-yCoords, -aoi_ymin)

            if xi0 >= xi1 or yi0 >= yi1:
                print(f" {key}: AOI does not overlap, skipping")
                continue

            # get the data from the s3 bucket
            hhhh = h5['/science/LSAR/GCOV/grids/frequencyA/HHHH']
            hvhv = h5['/science/LSAR/GCOV/grids/frequencyA/HVHV']

            # reduce the file sized by the stride
            if(stride != 1):
                hhhh = hhhh[::stride, ::stride]
                hvhv = hvhv[::stride, ::stride]

```

```

subsets[key] = {
    'date': row['date'],
    'track': row['track'],
    'frame': row['frame'],
    'HHHH': hhhh,
    'HVHV': hvhv,
    'xCoords': xCoords[xi0:xi1:stride],
    'yCoords': yCoords[yi0:yi1:stride],
}
print(f" {key}: shape={hhhh.shape}")

with np.errstate(divide='ignore', invalid='ignore'):
    data_db = 10 * np.log10(np.abs(hhhh))
    data_db[~np.isfinite(data_db)] = np.nan

if False:    # execute if you want to make some images
    fig, ax = plt.subplots(figsize=(10, 8))
    im = ax.imshow(data_db, cmap='gray',
                   vmin=np.nanpercentile(data_db, 2),
                   vmax=np.nanpercentile(data_db, 98))
    plt.colorbar(im, ax=ax, label='dB')
    ax.set_title(f"{aoi_name} - GCOV HHHH {row['date']} track")
    plt.tight_layout()
    plt.show()
    plt.close(fig)

all_subsets[aoi_name] = subsets
print(f" -> {len(subsets)} subsets for '{aoi_name}'")

print(f"\nDone. Total AOIs: {len(all_subsets)}")

```

AOI 'Gainesville' in EPSG:32617: x=[364161, 432351], y=[3252507, 3297446]

```

20251129_t098_f074: shape=(4494, 6819)
20251211_t098_f074: shape=(4494, 6819)
20251223_t098_f074: shape=(4494, 6819)
20260104_t098_f074: shape=(4494, 6819)
20260116_t098_f074: shape=(4494, 6819)
-> 5 subsets for 'Gainesville'

```

Done. Total AOIs: 1

```

In [9]: dynamic_range = 16
        hhmax, hhmin = -4, -4 - dynamic_range
        hvmax, hvmin = -8, -8 - dynamic_range

for aoi_name, subsets in all_subsets.items():
    print(f"\n=== {aoi_name} ===")
    for key, s in subsets.items():
        with np.errstate(divide='ignore', invalid='ignore'):
            hhhh_db = 10 * np.log10(np.abs(s['HHHH'])).astype(np.float

```



```

hvhv_db = 10 * np.log10(np.abs(s['HVHV'])).astype(np.float)

x_km = (s['xCoords'] - s['xCoords'][0]) / 1000.0
y_km = (s['yCoords'] - s['yCoords'][0]) / 1000.0
extent = [x_km[0], x_km[-1], y_km[-1], y_km[0]]

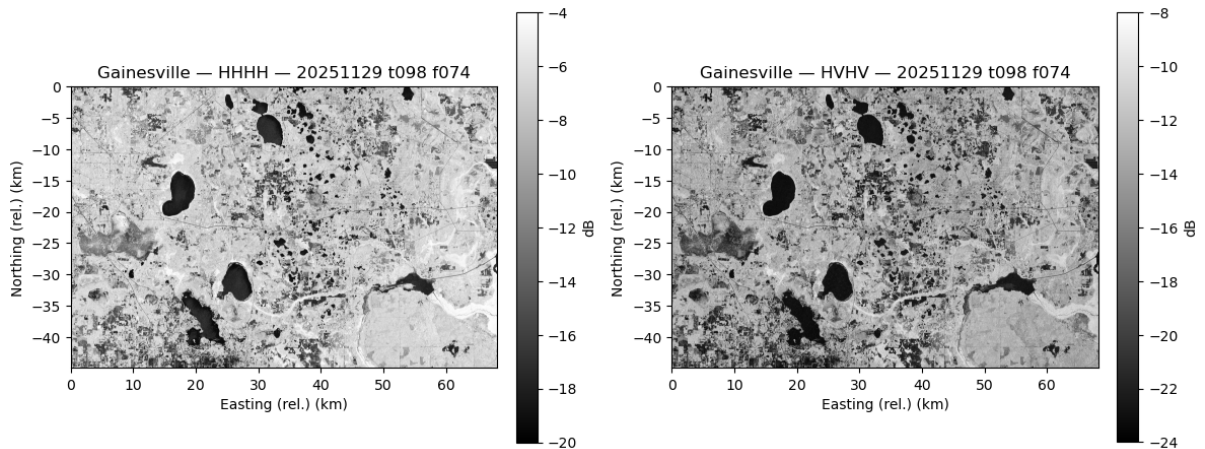
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
print(f"[{s['date']} track={s['track']} frame={s['frame']}]")

for ax, data, label, vmin, vmax in [
    (axes[0], hhhh_db, 'HHHH', hhmin, hhmax),
    (axes[1], hvhv_db, 'HVHV', hvmin, hvmax),
]:
    im = ax.imshow(data, cmap='gray', vmin=vmin, vmax=vmax, extent=extent)
    plt.colorbar(im, ax=ax, label='dB', fraction=0.046, pad=0.05)
    ax.set_title(f"{aoi_name} — {label} — {s['date']} t{s['track']} f{s['frame']}")
    ax.set_xlabel('Easting (rel.) (km)')
    ax.set_ylabel('Northing (rel.) (km)')

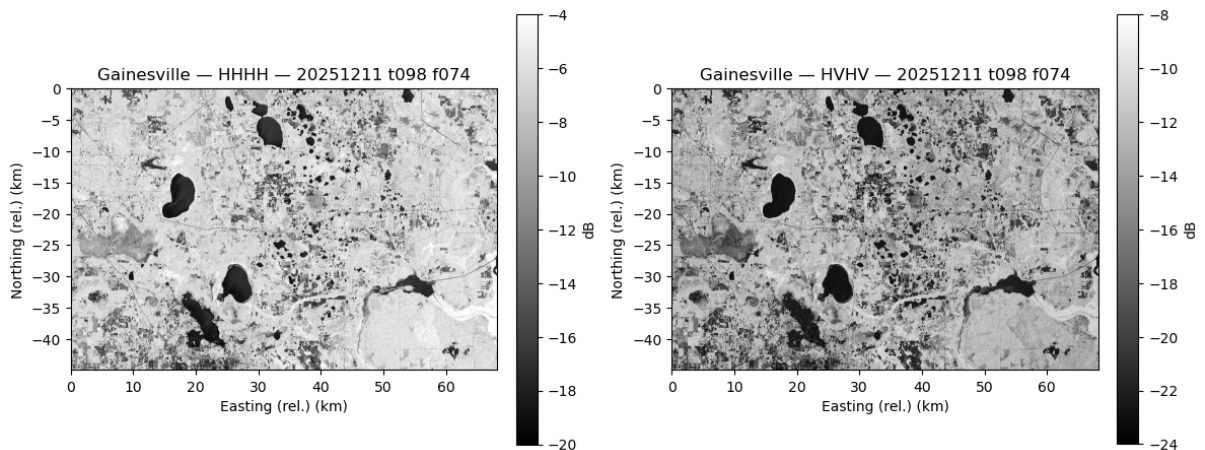
plt.tight_layout()
plt.show()
plt.close(fig)

```

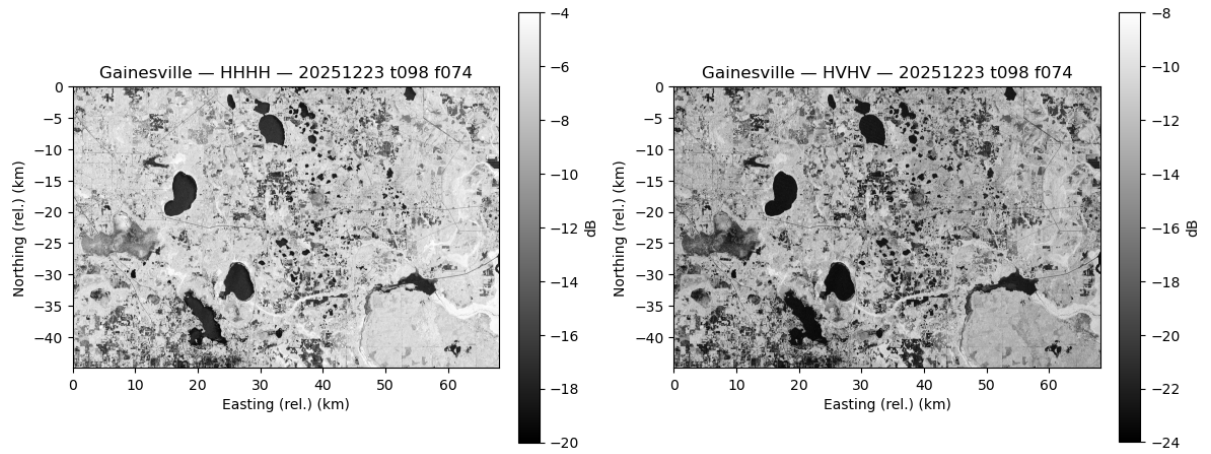
=== Gainesville ===
[20251129 track=098 frame=074]



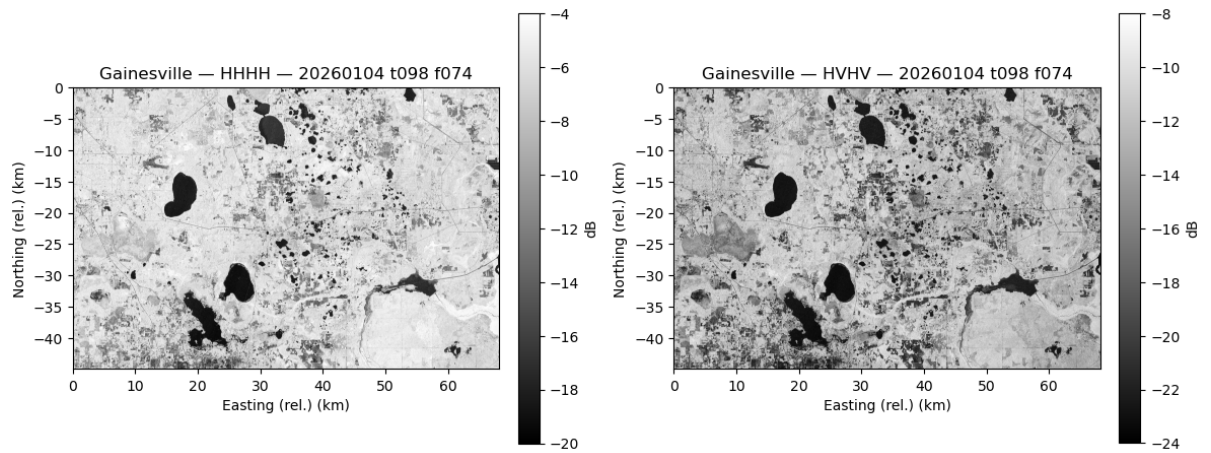
[20251211 track=098 frame=074]



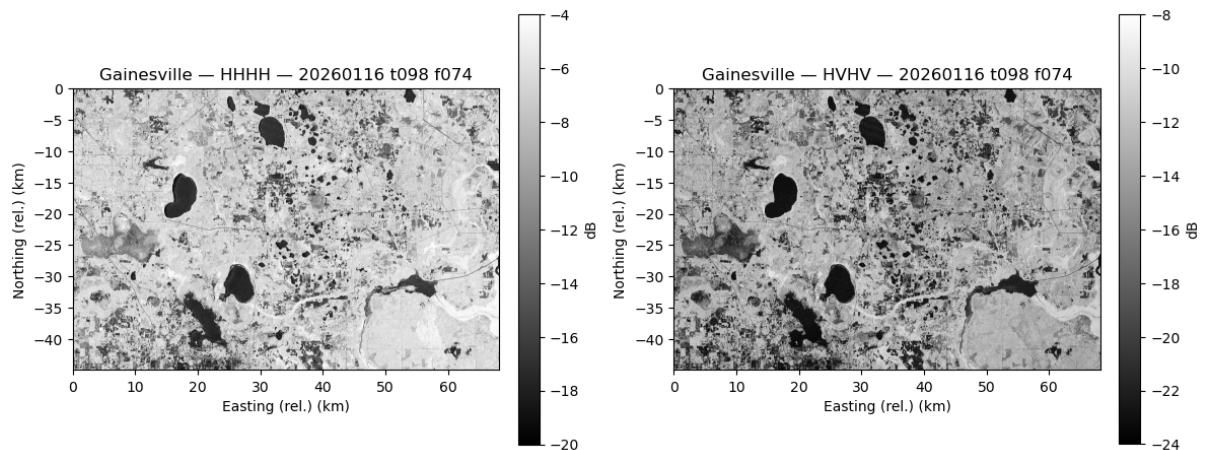
[20251223 track=098 frame=074]



[20260104 track=098 frame=074]



[20260116 track=098 frame=074]



In [10]: stats_by_aoi = {}

```
for aoi_name, subsets in all_subsets.items():
    print(f"\n=== Stacking: {aoi_name} ===")
    hhhh_stack = np.stack([s['HHHH'] for s in subsets.values()], axis=
    hvhv_stack = np.stack([s['HVHV'] for s in subsets.values()], axis=

    with np.errstate(divide='ignore', invalid='ignore'):
        hhhh_dBstack = 10 * np.log10(hhhh_stack)
        hvhv_dBstack = 10 * np.log10(hvhv_stack)
```



```

hhhh_dBstack[~np.isfinite(hhhh_dBstack)] = np.nan
hvhv_dBstack[~np.isfinite(hvhv_dBstack)] = np.nan

s0 = next(iter(subsets.values()))
dx = float(s0['xCoords'][1] - s0['xCoords'][0]) if len(s0['xCoords']) > 1 else 0
dy = float(s0['yCoords'][1] - s0['yCoords'][0]) if len(s0['yCoords']) > 1 else 0

stats_by_aoi[aoi_name] = {
    'hhhh_dBmedian': np.nanmedian(hhhh_dBstack, axis=0),
    'hvhv_dBmedian': np.nanmedian(hvhv_dBstack, axis=0),
    'hhhh_dBstd': np.nanstd(hhhh_dBstack, axis=0),
    'hvhv_dBstd': np.nanstd(hvhv_dBstack, axis=0),
    'hhhh_median': np.nanmedian(hhhh_stack, axis=0),
    'hvhv_median': np.nanmedian(hvhv_stack, axis=0),
    'xCoords': s0['xCoords'],
    'yCoords': s0['yCoords'],
    'dx': dx,
    'dy': dy,
}

print(f"    stack shape: {hhhh_stack.shape}")

print(f"\nStats computed for: {list(stats_by_aoi.keys())}")

```

```

=== Stacking: Gainesville ===
    stack shape: (5, 4494, 6819)

```

```
Stats computed for: ['Gainesville']
```

```

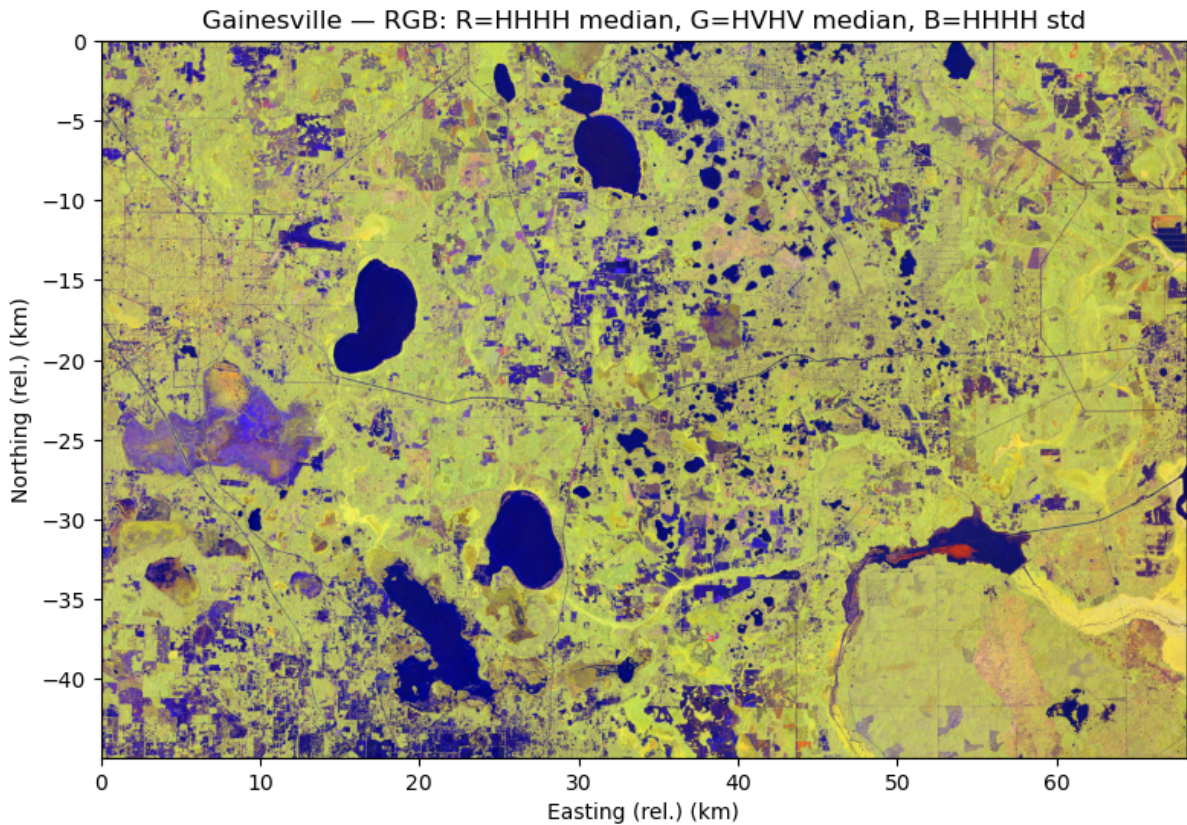
In [11]: def normalize(arr, pmin=2, pmax=98):
    lo, hi = np.nanpercentile(arr, pmin), np.nanpercentile(arr, pmax)
    return np.clip((arr - lo) / (hi - lo), 0, 1)

    for aoi_name, st in stats_by_aoi.items():
        rgb = np.dstack([
            normalize(st['hhhh_dBmedian']),
            normalize(st['hvhv_dBmedian']),
            normalize(st['hhhh_dBstd']),
        ])

        x_km = (st['xCoords'] - st['xCoords'][0]) / 1000.0
        y_km = (st['yCoords'] - st['yCoords'][0]) / 1000.0
        extent = [x_km[0], x_km[-1], y_km[-1], y_km[0]]

        fig, ax = plt.subplots(figsize=(8, 8))
        ax.imshow(rgb, extent=extent)
        ax.set_title(f"{aoi_name} - RGB: R=HHHH median, G=HVHV median, B=HHHH std")
        ax.set_xlabel('Easting (rel.) (km)')
        ax.set_ylabel('Northing (rel.) (km)')
        plt.tight_layout()
        plt.show()
        plt.close(fig)

```



```
In [12]: if True:
    targ_dir = '.'
    crs = CRS.from_epsg(EPSPG_keep)

    for aoi_name, st in stats_by_aoi.items():
        dx = st['dx']
        dy = st['dy']
        ulx = float(st['xCoords'][0]) - dx / 2.0
        uly = float(st['yCoords'][0]) - dy / 2.0
        transform = Affine(dx, 0.0, ulx, 0.0, dy, uly)

        h, w = st['hhhh_dBmedian'].shape
        profile = {
            "driver": "GTiff",
            "height": h,
            "width": w,
            "count": 1,
            "dtype": "float32",
            "crs": crs,
            "transform": transform,
            "compress": "deflate",
            "tiled": True,
            "blockxsize": 256,
            "blockysize": 256,
        }

        tag = aoi_name.replace(' ', '_')
        with rasterio.open(os.path.join(targ_dir, f"{tag}_HHHH.tif"),
```

```
dst.write(st['hhhh_dBmedian'].astype("float32"), 1)
with rasterio.open(os.path.join(targ_dir, f"{tag}_HVHV.tif"),
dst.write(st['hvhv_dBmedian'].astype("float32"), 1)
with rasterio.open(os.path.join(targ_dir, f"{tag}_HHstd.tif"),
dst.write(st['hhhh_dBstd'].astype("float32"), 1)
print(f"Saved GeoTIFFs for '{aoi_name}'")
```

Saved GeoTIFFs for 'Gainesville'